Lab. Getting Started with Rust

Overview

This lab exercise was written on an Ubuntu 22.04 instance. Some commands, such as package installation, may be different depending on the operating system in use, but otherwise these steps should work on other flavors of Linux with minimal edits.

The prompt of **root:~#** is used for simplicity in the labs; your prompt may look different depending on where you are running the commands, perhaps something like **root@LF-training-1-nyc1-ubuntu-22-04-x64-8399900:~#**

Install Rust

First we will install Rust. Open a terminal, become root if not already, and run the following command to install Rust using **rustup**, the official Rust installer. This command will download and run the Rust installer script. Follow the prompts to proceed with the installation. By default, **rustup** will add Rust to your system's **PATH** environment variable.

```
root:~#   curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
info: downloading installer

Welcome to Rust!

This will download and install the official compiler for the Rust
programming language, and its package manager, Cargo.

Rustup metadata and toolchains will be installed into the Rustup
home directory, located at:

  /root/.rustup
```

```
This can be modified with the RUSTUP_HOME environment variable.

The Cargo home directory is located at:

  /root/.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

  /root/.cargo/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

  /root/.profile
  /root/.bashrc

You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

Current installation options:


   default host triple: x86_64-unknown-linux-gnu
     default toolchain: stable (default)
               profile: default
  modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
>                   #<<-- Press Enter or Return for default installation
info: profile set to 'default'
info: default host triple is x86_64-unknown-linux-gnu
info: syncing channel updates for 'stable-x86_64-unknown-linux-gnu'
info: latest update on 2023-07-13, rust version 1.71.0 (8ede3aae2 2023-07-12)
info: downloading component 'cargo'
info: downloading component 'clippy'
info: downloading component 'rust-docs'
info: downloading component 'rust-std'
info: downloading component 'rustc'
info: downloading component 'rustfmt'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
```

```
 13.6 MiB /  13.6 MiB (100 %)   3.8 MiB/s in  3s ETA:  0s
info: installing component 'rust-std'
 25.4 MiB /  25.4 MiB (100 %)   9.8 MiB/s in  2s ETA:  0s
info: installing component 'rustc'
 64.0 MiB /  64.0 MiB (100 %)  11.0 MiB/s in  5s ETA:  0s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-x86_64-unknown-linux-gnu'

  stable-x86_64-unknown-linux-gnu installed - rustc 1.71.0 (8ede3aae2 2023-07-12)


Rust is installed now. Great!

To get started you may need to restart your current shell.
This would reload your PATH environment variable to include
Cargo's bin directory ($HOME/.cargo/bin).

To configure your current shell, run:
source "$HOME/.cargo/env"
```

After installing Rust, you need to update your terminal session to recognize the newly installed binaries. You can do this by running the following command in the terminal or by opening a new terminal session:

```
root:~#     source $HOME/.cargo/env
```

To verify that Rust is installed correctly, run the following command in the terminal. You should see the Rust version number if the installation was successful. Your version may be slightly different.

```
root:~#     rustc --version
rustc 1.71.0 (8ede3aae2 2023-07-12)
```

It's good practice to keep Rust up-to-date. To update Rust and its components, use the following command:

```
root:~#     rustup update
info: syncing channel updates for 'stable-x86_64-unknown-linux-gnu'
info: checking for self-update

  stable-x86_64-unknown-linux-gnu unchanged - rustc 1.71.0 (8ede3aae2 2023-07-12)

info: cleaning up downloads & tmp directories
```

Get Started with Cargo

Cargo is the build system and package manager for Rust. It comes bundled with the Rust installation. You can use it to create new Rust projects, manage dependencies, and build your code.

Begin by learning the options and arguments to Cargo.

```
root:~#     cargo -h
Rust's package manager

Usage: cargo [+toolchain] [OPTIONS] [COMMAND]

Options:
  -h, --help                Print help
  -V, --version             Print version info and exit
      --list                List installed commands
      --explain <CODE>      Run `rustc --explain CODE`
  -v, --verbose...          Use verbose output (-vv very verbose/build.rs output)
  -q, --quiet               Do not print cargo log messages
      --color <WHEN>        Coloring: auto, always, never
  -C <DIRECTORY>            Change to DIRECTORY before doing anything
(nightly-only)
      --frozen              Require Cargo.lock and cache are up to date
      --locked              Require Cargo.lock is up to date
      --offline             Run without accessing the network
      --config <KEY=VALUE>  Override a configuration value
  -Z <FLAG>                 Unstable (nightly-only) flags to Cargo, see 'cargo -Z
help' for details

Some common cargo commands are (see all commands with --list):
    build, b    Compile the current package
    check, c    Analyze the current package and report errors, but don't build
object files
    clean       Remove the target directory
<output_omitted>
```

Create a new project. Notice the case of LF.

```
root:~#     cargo new my_LF_project
     Created binary (application) `my_LF_project` package
```

Change to the project directory you created using Cargo:

```
root:~#     cd my_LF_project
```

Take a look at the files created:

```
root:~#      ls -R
.:
Cargo.toml  src

./src:
main.rs

root:~#    cat Cargo.toml
[package]
name = "my_LF_project"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at
https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
```

There is also a `main.rs` file created, which is one of the file names Rust will look for during compilation.

```
root:~#    cat src/main.rs
fn main() {
    println!("Hello, open source world!");
}
```

Then, you can try to build and run your Rust project using Cargo. Note you may get the following error, depending on what software you have installed. We will assume you don't have a compiler. We also illustrate that the name must follow a particular syntax.

```
root:~#     cargo build
   Compiling my_LF_project v0.1.0 (/root/my_LF_project)
warning: crate `my_LF_project` should have a snake case name
  |
  = help: convert the identifier to snake case: `my_lf_project`
  = note: `#[warn(non_snake_case)]` on by default

error: linker `cc` not found
  |
  = note: No such file or directory (os error 2)

warning: `my_LF_project` (bin "my_LF_project") generated 1 warning
error: could not compile `my_LF_project` (bin "my_LF_project") due to previous
error; 1 warning emitted
```

Take a look at the directory again, and notice there is a new subdirectory and a lock file. View the contents of both.

```
root:~#    ls -R
.:
Cargo.lock  Cargo.toml  src  target

./src:
main.rs

./target:
CACHEDIR.TAG  debug

./target/debug:
build  deps  examples  incremental

./target/debug/build:

./target/debug/deps:
my_LF_project-cae019c4190ab598.2xpekdmz37adtgu3.rcgu.o
my_LF_project-cae019c4190ab598.4tizn615crk9ikdz.rcgu.o
my_LF_project-cae019c4190ab598.d
my_LF_project-cae019c4190ab598.3504268gvqe7567k.rcgu.o
my_LF_project-cae019c4190ab598.504u6fhp5f4qhq1t.rcgu.o
my_LF_project-cae019c4190ab598.tn1imvw2e6flzhx.rcgu.o
my_LF_project-cae019c4190ab598.36idghvq7964bmb.rcgu.o
my_LF_project-cae019c4190ab598.576ayijqcamb6d5u.rcgu.o

./target/debug/examples:

./target/debug/incremental:
my_LF_project-3is89aji6sugd

./target/debug/incremental/my_LF_project-3is89aji6sugd:
s-gn6zf3atnf-fmx2cc-8xtq5u0o2xgj0b35hza6xkv0f  s-gn6zf3atnf-fmx2cc.lock

./target/debug/incremental/my_LF_project-3is89aji6sugd/s-gn6zf3atnf-fmx2cc-8xtq5u
0o2xgj0b35hza6xkv0f:
2xpekdmz37adtgu3.o  3504268gvqe7567k.o  36idghvq7964bmb.o  504u6fhp5f4qhq1t.o
576ayijqcamb6d5u.o  dep-graph.bin  query-cache.bin  tn1imvw2e6flzhx.o
work-products.bin

root:~#      cat Cargo.lock
# This file is automatically @generated by Cargo.
# It is not intended for manual editing.
version = 3

[[package]]
name = "my_LF_project"
version = "0.1.0"
```

From the previous errors we see a warning that we need to change the name; we also see a cc linker error that we do not have a compiler installed. You may not see this error if you had already installed a compiler on your system.

```
root:~#     apt update
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Hit:2 http://mirrors.digitalocean.com/ubuntu jammy InRelease
Get:3 http://mirrors.digitalocean.com/ubuntu jammy-updates InRelease [119 kB]
Get:4 http://mirrors.digitalocean.com/ubuntu jammy-backports InRelease [108 kB]
Get:5 http://mirrors.digitalocean.com/ubuntu jammy-updates/main amd64 Packages
[853 kB]
Get:6 http://mirrors.digitalocean.com/ubuntu jammy-updates/main Translation-en
[208 kB]
<output_omitted>
```

During installation of the package you will see a CLI screen asking which services should be restarted. Use **<Tab>** to move the highlight to **Ok**, and continue with the default services selected.

```
root:~#      apt install build-essential -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bzip2 cpp cpp-11 dpkg-dev fakeroot fontconfig-config fonts-dejavu-core g++
g++-11 gcc gcc-11 gcc-11-base
<output_omitted>
```

To avoid the naming convention error, edit the `Cargo.toml` file and change the upper case letters to lower case. This example uses vim as the editor, but any text editor such as nano or emacs would work.

```
root:~#     vim Cargo.toml
[package]
name = "my_lf_project"                #<--  Change to lower-case lf
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at
https://doc.rust-lang.org/cargo/reference/manifest.html

[dependencies]
```

Now try to build the crate again. There should be no errors this time.

```
root:~#     cargo build
   Compiling my_lf_project v0.1.0 (/root/my_LF_project)
    Finished dev [unoptimized + debuginfo] target(s) in 0.18s
```

With a successful build we can test that the program runs.

```
root:~#     cargo run
   Compiling my_lf_project v0.1.0 (/root/my_LF_project)
    Finished dev [unoptimized + debuginfo] target(s) in 0.18s

Hello, open source world!
```

A first step for troubleshooting is to use the **-v** or **-vv** arguments to raise the verbosity of the output. In this case, there is only an extra line of output.

```
root:~#     cargo run -v
       Fresh my_lf_project v0.1.0 (/root/my_LF_project)
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
     Running `target/debug/my_lf_project`
Hello, world!
```

Take a look at the files which have been created. Notice there is a new **target** and **target/debug** directory created, inside of which is a new binary **my_lf_project**.

```
root:~#     ls -R
   .:
Cargo.lock  Cargo.toml  src  target

./src:
main.rs

./target:
CACHEDIR.TAG  debug

./target/debug:
build  deps  examples  incremental  my_lf_project  my_lf_project.d

./target/debug/build:
<output_omitted>
```

Run the binary and see if it behaves as expected.

```
root:~#     ./target/debug/my_lf_project
Hello, world!
```

Format Code

There are several opinions on the best way to format code for readability, usually centered around personal experience and preference. The **rustfmt** command can be used to show and or update formatting.

Begin by looking at some of the options of **rustfmt**:

```
root:~#    rustfmt -h
Format Rust code

usage: rustfmt [options] <file>...

Options:
        --check         Run in 'check' mode. Exits with 0 if input is
                        formatted correctly. Exits with 1 and prints a diff if
                        formatting is required.
        --emit [files|stdout]
                        What data to emit and how
        --backup        Backup any modified files.
<output_omitted>
```

View configuration options available:

```
root:~#    rustfmt -h config
Configuration Options:
                          max_width <unsigned integer> Default: 100
                                    Maximum width of each line

                          hard_tabs <boolean> Default: false
                                    Use tab characters for indentation, spaces
for alignment

                         tab_spaces <unsigned integer> Default: 4
                                    Number of spaces per tab
<output_omitted>
```

Locate the `format.rs` file in the course tarball. View the file to see the non-standard formatting.

```
root:~#    cat format.rs
fn main() { println!("Hello, open source world!");
println!("Each line has different things");

// Comments    have     lots of     space
//
//
                  println!("20 spaces  in  and    has      lots     of     space
");
}
```

View what would be changed by running **rustfmt**. Verify that no change has happened to the original file afterwards.

```
root:~#    rustfmt format.rs --emit stdout
/root/format.rs:

fn main() {
    println!("Hello, open source world!");
    println!("Each line has different things");

    // Comments    have     lots of     space
    //
    //
    println!("20 spaces  in  and    has      lots     of     space ");
}

root:~#    cat format.rs
<output_omitted>
```

Now update the formatting of the file and verify the changes are persistent in the file.

```
root:~#    rustfmt format.rs -v
Formatting /root/format.rs
Spent 0.000 secs in the parsing phase, and 0.000 secs in the formatting phase
```

Check the formatting of the file. It should follow generally accepted formatting guidelines.

```
root:~#    cat format.rs
fn main() {
    println!("Hello, open source world!");
    println!("Each line has different things");

    // Comments    have     lots of     space
    //
    //
    println!("20 spaces  in  and    has      lots     of     space ");
}
```

Basic Troubleshooting Tools

There are several tools to use when debugging and troubleshooting your code. We will get started with the basics of Clippy and then Rust Analyzer, but you will probably want to spend time exploring new tools, as well as these tools to more depth.

Using Clippy for Linting

Clippy may already be installed as a component; check that it is and is also up to date.

```
root:~# rustup component add clippy-preview
info: component 'clippy' for target 'x86_64-unknown-linux-gnu' is up to date
```

Locate, download and extract the course tarball. Inside you should see a directory called `clippy1`. Change into that directory.

```
root:clippy1# cargo clippy -h
Checks a package to catch common mistakes and improve your Rust code.

Usage:
    cargo clippy [options] [--] [<opts>...]

Common options:
    --no-deps               Run Clippy only on the given crate, without linting
the dependencies
    --fix                   Automatically apply lint suggestions. This flag
implies `--no-deps` and `--all-targets`
<output_omitted>
```

Check the current crate for errors. You should see one indicating that a letter is found instead of a sequence of numbers. Note there is a **`rustc`** command to look up the error. After replacing with the typical number, run **`cargo clippy`** again and keep making changes until there are no errors.

```
root:clippy1#  cargo clippy
    Checking clippy1 v0.1.0 (/root/clippy1)
error[E0425]: cannot find value `B` in this scope
 --> src/main.rs:2:27
  |
2 |     let mut vec = vec![1, B, 3, 4, 5];
  |                           ^ not found in this scope

For more information about this error, try `rustc --explain E0425`.
error: could not compile `clippy1` (bin "clippy1") due to previous error
```

Get Started with Rust Analyzer

Using Rust Analyzer involves integrating it with your code editor or IDE to leverage its real-time code analysis and intelligent features. Rust Analyzer provides rich support for Rust code, including autocompletion, in-place diagnostics, and various code suggestions. Here's a step-by-step guide to using Rust Analyzer:

View all the commands which begin with **`rust`**. Use the **`Tab`** key twice to see commands which match the starting stub.

```
root:clippy1#  rust<Tab>-<Tab>
```

```
<output varies depending on which packages have been installed>
```

Run the **rust-analyzer** command. You should see an error as we have not yet installed the software.

```
root:clippy1#  rust-analyzer
error: 'rust-analyzer' is not installed for the toolchain
'stable-x86_64-unknown-linux-gnu'
```

Use **rustup** to add the component:

```
root:clippy1#  rustup component add rust-analyzer
info: downloading component 'rust-analyzer'
info: installing component 'rust-analyzer'
```

View the options. The output may change, but should be close to the output seen below.

```
root:clippy1#  rust-analyzer -h
rust-analyzer
  LSP server for the Rust programming language.

  Subcommands and their flags do not provide any stability guarantees and may be
removed or
  changed without notice. Top-level flags that are not are marked as [Unstable]
provide
  backwards-compatibility and may be relied on.

OPTIONS:
    -v, --verbose
<output_omitted>
```

Run the command again. Use the **analysis-stats** argument and pass the current directory which has the `Cargo.toml` file.

```
root:clippy1#  rust-analyzer analysis-stats .
Database loaded:     613.33ms, 62minstr (metadata 455.17ms, 532kinstr; build
93.69ms, 78kinstr)
  crates: 1, mods: 1, decls: 1, fns: 1
Item Collection:     3.59s, 22ginstr
  exprs: 10, ??ty: 0 (0%), ?ty: 0 (0%), !ty: 0
  pats: 0, ??ty: 0 (100%), ?ty: 0 (100%), !ty: 0
Inference:           1.53s, 8421minstr
Total:               5.12s, 30ginstr
```

Run the command again, this time view the Language Server Index Format (LSIF) output.

```
root:clippy1# rust-analyzer lsif .
Generating LSIF started...
```

{"id":0,"type":"vertex","label":"metaData","version":"0.5.0","projectRoot":"file:///root/my_LF_project","positionEncoding":"utf-16","toolInfo":{"name":"rust-analyzer","version":"1.71.0 (8ede3aa 2023-07-12)"}}
{"id":1,"type":"vertex","label":"document","uri":"file:///root/my_LF_project/src/main.rs","languageId":"rust"}
{"id":2,"type":"vertex","label":"foldingRangeResult","result":[{"startLine":0,"startCharacter":10,"endLine":2,"endCharacter":1}]}
<output_omitted>

Explore using the commands you have learned on some of the files included in the course tarball as time allows. You may have to create new crates and copy the file to replace the `src/main.rs` file in order to build and test the file.